

L12 评估体系 总结

L12 评估体系 · 课程总结

封版日期：2026-08-04 · 对标书 Ch6 ·  融合课 (JD + 书) 。交付物： `eval/rubric_judge.py` (Rubric-based LLM-judge + 金标集 + 校准) + `eval/test_rubric_judge.py` (4 测试) 。定位： **收束课**——把 L7 eval 集、reflect/代码审查的 judge、Langfuse tracing 收成体系。

一、这节课学了什么（一句话）

评估不是“跑个准确率”，是一套体系：**分层指标 + 结构化 Rubric 的 LLM-as-Judge（校准过的） + 统计显著性（别被噪声骗） + 可观测性回流的活评估集。**

二、指标词典（分层）

- **过程**：行动合法率、**工具调用正确率**(JD)、路径效率(步数/回退)、**成本延迟**(JD, = Langfuse 看到的)。
- **结果**：任务成功率 +  **Pass@k vs Pass^k**:
 - Pass@k = 至少一次成功（能力上限）；Pass^k = 全部成功（稳定性）。
 - 单次 80% → Pass@3≈99% 但 **Pass^3≈51%**。回归测试用 Pass^k (Pass@k 掩盖不稳定)。
 - 公式假设 k 次独立同分布；现实失败相关，真实 Pass^k 要**实测**。
- **安全**：一票否决（零容忍）。
-  **轨迹 vs 结果双重覆盖**：说“退款成功”（轨迹）vs 账上真退了(结果)——L5「看动作不看话术」体系化。

三、LLM-as-a-Judge

- **局限**：长度偏差（偏爱长回复，会被 reward hacking 利用）、评判波动。防范：Rubric 惩罚冗长 + 审计分数-长度相关性 + 配对时对齐长度。
- **Rubric 四准则** (Scale AI)：① 专家指导 ② 全面覆盖+陷阱 ③ **权重+一票否决(veto)** ④ 自包含可验证（避免“展示了深刻理解”，改成“引用≥2理论”）。
-  **评判者校准（放量前必做）**：金标集(100-200 人工标注) → 测 judge vs 人类一致率(**Cohen' s kappa > 0.7**)才放量。未校准的 judge 分只是“另一个模型的意见”。
- **同源模型盲区** → 多源评判（回扣 L11 判据 + reflect 同模型自审）。

四、统计显著性 (senior 分水岭)

- 二项标准误 $\sqrt{p(1-p)/n}$: 100 用例 70% $\rightarrow$ SE $\approx$ 4.6% $\rightarrow$ CI $\approx$ 70% $\pm$ 9pp $\rightarrow$ “73% vs 70%” 在噪声里。
- 多次运行取均值(3-5 次种子) + 配对分析(McNemar)(扣题目难易噪声) + 防多重比较(6 假设 $\rightarrow$ 26% 假阳性)。
- 铁律: 分差 < 噪声带宽 $\rightarrow$ 不切换; 小评估集分辨不出小改进 $\rightarrow$ 先扩评估集。

五、可观测性 = 评估的数据地基 (= 刚做的 Langfuse)

- Trace/span 树 (OpenTelemetry/OpenInference) ; 异步批量采集不影响延迟。
- ★ 回流成评估资产: 生产失败案例 $\rightarrow$ 脱敏 $\rightarrow$ 评估集新回归用例。评估集从静态变活资产。

六、动手: Rubric-based LLM-judge (填掉 L7 子串匹配的坑)

背景: L7 eval 用 `must_not_contain` (子串匹配) 判红线, `_meta` 早写明 “L12 升级为 LLM-as-judge”。PROGRESS 记过三次撞上 “子串区分不了确认与否认”。

做的 (`eval/rubric_judge.py`) : - **RUBRIC** : 多维度 + **veto 一票否决** (幻觉/确认假前提/越权泄露 $\rightarrow$ 判 fail, 不管其他维度) 。 - `judge_reply(question, reply, expected_points, client)` : LLM-judge, 结构化输出, 依赖注入。 - **GOLDEN_SET** (6 条人工判定) + `calibrate()` : 跑 judge 对比人类, 算一致率。

★ 高光证据 (赢过子串匹配) : 同一个 “100 天无理由” 假前提——|| g03 「不是, 是7天」 | g04 「是的, 100天」 ||——||——|| 都含子串 **100天** |  |  || 子串匹配 `must_not_contain` |  都误杀 |  || **LLM-judge** | **pass** (懂否认) | **fail** (懂确认) |

校准: 金标集 6/6 一致 (100%) 。但诚实局限: 6 条统计上不够, 真实要 100-200 条 + **kappa** (回扣 \$4, 自己打脸自己的 demo) 。

七、这次踩的坑 (真实工程故事)

1. `json.loads` 结果不保证是 **dict** (用户自己抓到) : 可能是 list/str/数字, 或缺 key $\rightarrow$ 要**校验形状** (isinstance dict + 必需 key 在不在) , 不能光 `except JSONDecodeError` 。
2. **raise/except 类型不匹配**: `raise ValueError` 却 `except json.JSONDecodeError` $\rightarrow$ 接不住 (JSONDecodeError 是 ValueError 的**子类**, catch 子类接不住父类) $\rightarrow$ 改 `except (JSONDecodeError, ValueError)` 。
3. **veto/verdict 归一化** (差点漏, 多部分只做一半) : 模型自相矛盾(veto=True 但 verdict=pass)时, 代码强制 verdict=fail, 别信模型。
4. **judge 兜底方向**: 选 fail-closed (红线场景保守) ; 但更精准的 eval 工程答案是**单独记 error**, 别把 “评委故障” 混进 agent 的 pass/fail 统计。

八、门禁三条 (全过)

- 环境可复现  (uv)
- 4 测试正反俱全  : veto 归一化 / fail-closed 兜底 / 正常 pass / calibrate 计数。

- 无残留 ☒ (你来写·TODO 清光、py_compile 过、client 注入无硬编码 key)。

九、Quiz (5/5)

子串匹配的正解=语义 LLM-judge (确定性用工具、语义用 judge) · Pass@3≈99%/Pass^3≈51% 回归用 Pass^k · 长度偏差 · 30 条 80%→87% 不能上线(噪声+藏回归,配对分析) · “模型强≠judge 可信” ,放量前校准。

一句话简历版：把 agent 评估从关键词匹配升级为**校准过的 Rubric LLM-as-Judge** (含一票否决维度) , 能语义区分 “确认 vs 纠正假前提” (子串匹配做不到) ; 并用金标集测 judge 与人类一致率、用二项标准误/配对分析判断改动是否真的显著——不被小样本噪声骗。